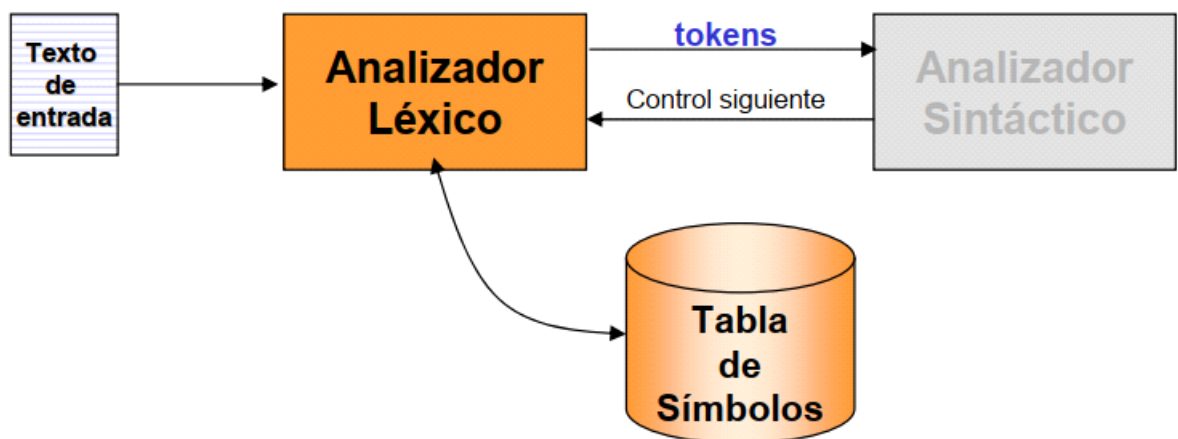


Tema 2 - Análisis de Léxico

Profesor Ramón López-Cózar Delgado, Curso 21-22

2.1 Descripción Funcional:

- Un analizador léxico lee, carácter a carácter, del documento de entrada (texto fuente) y genera una secuencia de patrones léxicos denominados tokens, y en su caso, asocia atributos a los tokens.
- El analizador crea la tabla de símbolos y la rellena.



- Aportaciones del analizador léxico
 - Simplifica el diseño del analizador sintáctico, confiriéndole una mayor eficacia.
 - Permite la portabilidad del traductor.

2.2 Conceptos básicos:

- **Token:** Conjunto de secuencias de caracteres con la misma misión sintáctica.
- **Lexema:** Secuencia de caracteres que forma un token.
- **Patrón:** Regla o reglas que describen a los lexemas asociados a un token, esto es una expresión regular.

Ejemplo 2.1: Dada la especificación de este lenguaje.

```
P → S | P S
S → repeat exp S;
   while exp S;
   if exp S;
   id = exp;
exp → exp * exp
    exp / exp
    exp + exp
    + exp
    * exp
    (exp)
    constante
    id
```

Inicialmente, tomamos los siguientes lexemas: **repeat**, **while**, **if**.
¿qué tienen en común?... Forman parte de una sentencia **S** y van seguidos de "**exp S;**"

COINCIDEN EN TODO LO QUE LE PRECEDE Y SUCEDE EN UN TEXTO

Es decir, es posible agrupar estos tres lexemas en otro de mayor abstracción para constituir un **token**.

Consecuencia: Reducimos tres reglas gramaticales en una sola:

$$S \rightarrow \text{CONDICIO} \text{ exp } S ;$$
$$| \text{ id } = \text{ exp } ;$$

Donde, **CONDICIO** es un token definido por el siguiente patrón:

("repeat" | "while" | "if")

- **Importante:** La abstracción léxica debe hacerse correctamente, por ejemplo los símbolos **+**, **/** y ***** no pueden ser un mismo lexema porque el **+** tiene una versión unaria, junto con *****, el símbolo **/** no lo posee, por lo tanto no poseen la misma función sintáctica.

2.3 Fundamentos: Álgebra de Lenguajes

- **Alfabeto:** Es un conjunto de símbolos. $A = a, b, \dots, z, A, B, \dots Z$
- **Lenguaje:** Conjunto de secuencias de símbolos de un alfabeto, a un alfabeto A le corresponde un

lenguaje elemental que es $A. L = a, b, \dots, z, A, B, \dots Z$.

Operaciones con lenguajes

- **Unión:** $L_1 \cup L_2 = \{x/x \in L_1 \text{ o } x \in L_2\}$
- **Intersección:** $L_1 \cap L_2 = \{x/x \in L_1 \text{ y } x \in L_2\}$
- **Concatenación:** $L_1 L_2 = \{xz/x \in L_1 \text{ y } z \in L_2\}, L^0 = \{\lambda\}$
- **Clausura:** $L^* = \bigcup_{i=0}^{\infty} L^i$
- **Semigrupo:** $L^+ = \bigcup_{i=1}^{\infty} L^i$

Formas de especificar un lenguaje

Por Extensión: $L = \{a, aa, aba, abbaa, \dots\}$

Por Comprensión: **GRAMÁTICA**

Especificación de tokens y patrones

- Esta tabla es lo que el analizador léxico le provee al sintáctico, tiene una forma similar a la siguiente:
 - En el analizador sintáctico los atributos no se utilizarán como tal, pero en el semántico si tiene su uso para determinar si es correcto.

Token	Código	Atributos	Patrón/Expresión Regular
CONDICLO	257	0: repeat, 1: while, 2: for	("repeat" "while" "if")

Expresiones regulares

- Dado un alfabeto Σ , las reglas que definen las expresiones regulares de Σ son:
 - λ es una expresión regular que representa la secuencia vacía: $\{\lambda\}$
 - Si $a \in \Sigma$ entonces $\{a\}$ es una expresión regular.
 - Sean r y s dos expresiones regulares representadas por los lenguajes $L(r)$ y $L(s)$. Se cumple:
 - $(r)|(s)$ es una expresión regular representada por $L(r) \cup L(s)$.
 - $(r)(s)$ es una expresión regular representada por $L(r)L(s)$.
 - $(r)^*$ es una expresión regular representada por $L(r)^*$.
 - (r) es una expresión regular representada por $L(r)$.

2.3.4 Propiedades algebraicas de las expresiones regulares

Con las definiciones anteriores se cumplen las siguientes propiedades:

- Conmutativa: $r|s = s|r$.
- Asociativa: $r|(s|t) = (r|s)|t$.
- Asociativa sobre la concatenación: $(rs)t = r(st)$.
- Distributiva de la concatenación y unión: $r(s|t) = rs|rt$ y $(r|t)s = rs|ts$.
- Elemento neutro: $\lambda r = r$ y $r\lambda = r$, siendo λ el elemento neutro y también se puede expresar $r^* = (r|\lambda)^*$ y $r^{**} = r^*$.

También se define la siguiente notación taquigráfica para la representación de expresiones regulares:

1. Uno o más veces: operador $+$, $(r^+ = r^+|\lambda)$.
2. Cero o una vez: operador $?$.
3. Cero o más veces: operador $*$.
4. Una forma cómoda de definir *clases de caracteres* es de la siguiente forma:

$$a|b|c|\dots|z = [a-z]$$

Autómata Finito no Determinista (AFN) asociado a una expresión regular: Construcción de Thompson

- **Teorema:** A toda expresión regular le corresponde un AFN.
- La construcción de Thompson permite obtener el AFN asociado a una expresión regular dada.

- Ejemplos:

1. Para la secuencia vacía λ

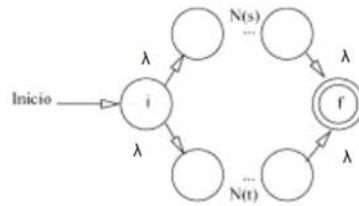


2. Para la secuencia $a \in \Sigma$

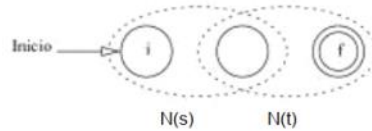


3. Sean s y t expresiones regulares y $N(s)$ y $N(t)$ los AFNs correspondientes, tenemos:

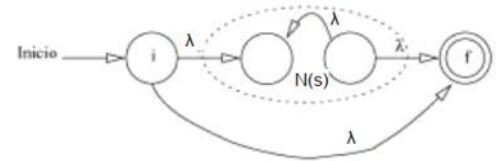
(a) Para $s|t$



(b) Para st



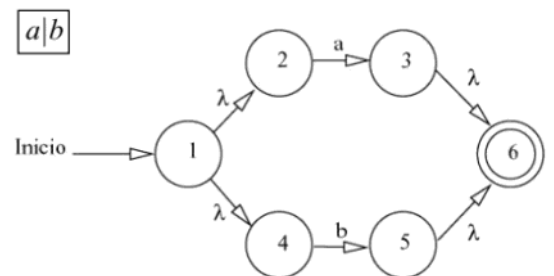
(c) Para expresiones s^* se construye el AFN $N(s^*)$

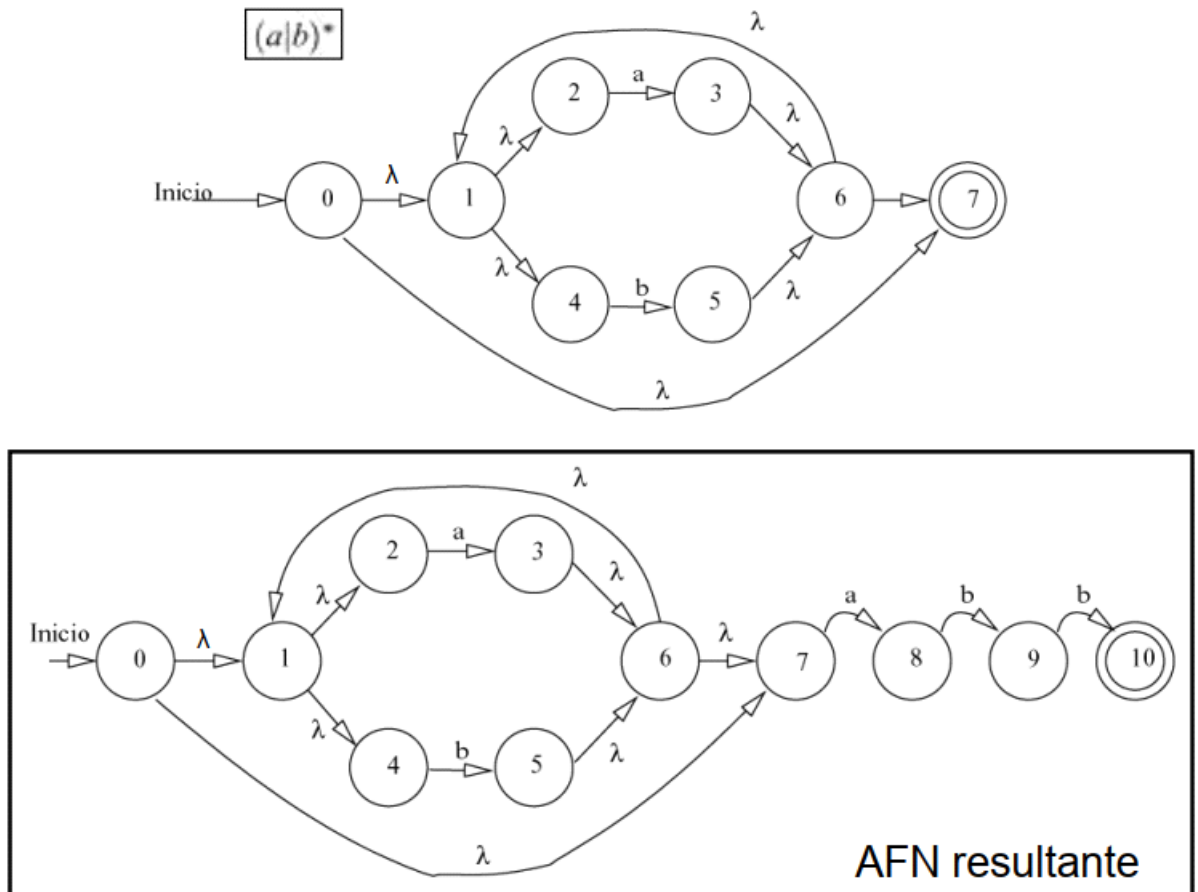


Supongamos la siguiente expresión regular

$$r = (a|b)^*abb$$

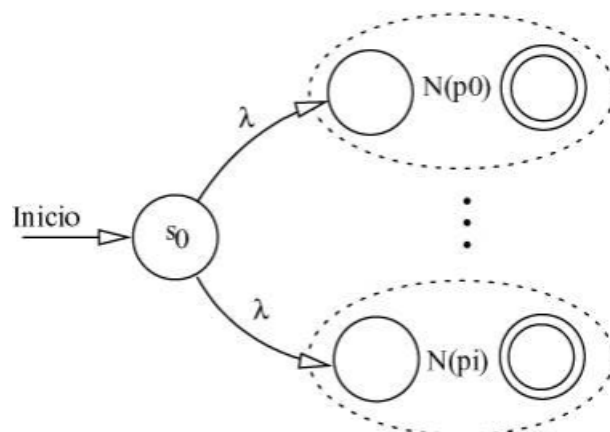
La forma de construir el AFN es por pasos. En primer lugar el de los símbolos a y b por separado:





AFN reconocedor de patrones

- Sean $p_0 | p_1 | p_2 | \dots | p_n$ los patrones a reconocer, se le asocia a cada patrón un AFN, añadiendo para cada patrón una λ -transición desde un estado inicial.



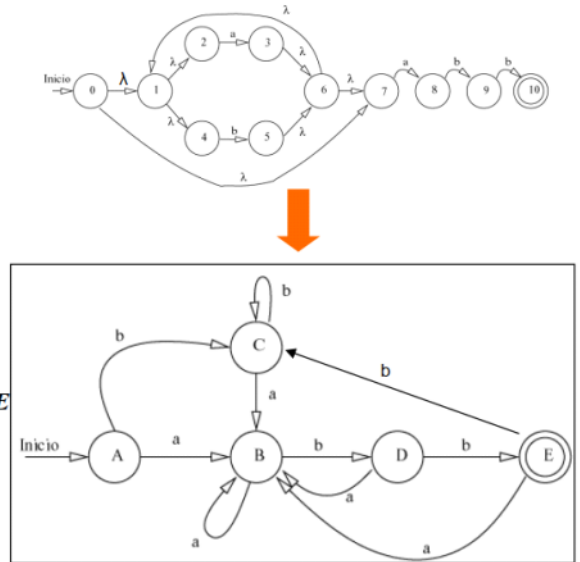
Autómata Finito Determinista (AFD) sin y con anticipación

- Existe un teorema en Teoría de Autómatas por el que a todo AFN se le puede asociar un Autómata Finito Determinista (AFD) equivalente.
- Un AFD es un caso especial de AFN en el que se cumplen las siguientes condiciones:
 - Ningún estado tiene λ -transiciones.
 - Para cada estado s y cada símbolo de entrada a , hay a lo sumo una arista etiquetada a que sale de s .
- Para la conversión se emplean tres funciones:
 - λ -transición (s): Conjunto de estados del AFN alcanzables desde el estado s únicamente con λ -transiciones.

- λ -clausura (T): Conjunto de estados del AFN alcanzables desde algún estado s en T únicamente con λ -transiciones, siendo T un conjunto de estados.
- $\text{Move}(T, a)$: Conjunto de estados del AFN hacia los cuales hay una transición con el símbolo de entrada de a en algún estado s en T .
- Si un estado e del AFD contiene algún estado final del AFN, entonces e es un estado final del ADF.

Ejemplo 2.2: Construcción del AFD equivalente a partir del AFN anterior. El alfabeto de entrada es $\{a, b\}$.

$\lambda\text{-clausura}(0) = \{0, 1, 2, 4, 7\} = A$
 $\lambda\text{-clausura}(\text{Move}(A, a)) = \lambda\text{-clausura}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\} = B$
 $\text{Trans}(A, a) = B$
 $\lambda\text{-clausura}(\text{Move}(A, b)) = \lambda\text{-clausura}(\{5\}) = \{1, 2, 4, 5, 6, 7\} = C$
 $\text{Trans}(A, b) = C$
 $\lambda\text{-clausura}(\text{Move}(B, a)) = \lambda\text{-clausura}(\{3, 8\}) = B$
 $\text{Trans}(B, a) = B$
 $\lambda\text{-clausura}(\text{Move}(B, b)) = \lambda\text{-clausura}(\{5, 9\}) = \{1, 2, 4, 5, 6, 7, 9\} = D$
 $\text{Trans}(B, b) = D$
 $\lambda\text{-clausura}(\text{Move}(C, a)) = \lambda\text{-clausura}(\{3, 8\}) = B$
 $\text{Trans}(C, a) = B$
 $\lambda\text{-clausura}(\text{Move}(C, b)) = \lambda\text{-clausura}(\{5\}) = C$
 $\text{Trans}(C, b) = C$
 $\lambda\text{-clausura}(\text{Move}(D, a)) = \lambda\text{-clausura}(\{3, 8\}) = B$
 $\text{Trans}(D, a) = B$
 $\lambda\text{-clausura}(\text{Move}(D, b)) = \lambda\text{-clausura}(\{5, 10\}) = \{1, 2, 4, 5, 6, 7, 10\} = E$
 $\text{Trans}(D, b) = E$
 $\lambda\text{-clausura}(\text{Move}(E, a)) = \lambda\text{-clausura}(\{3, 8\}) = B$
 $\text{Trans}(E, a) = B$
 $\lambda\text{-clausura}(\text{Move}(E, b)) = \lambda\text{-clausura}(\{5\}) = C$
 $\text{Trans}(E, b) = C$



- **Implementación del Analizador de Léxico:**
 - Autómata reconocedor de patrones: AFD con anticipación
 - Esto es lo que utiliza LEX, con la anticipación no es necesario usar las palabras reservadas.
 - Autómata reconocedor de palabras: AFD + Tabla de palabras reservadas
 - Permite al analizador saber si la palabra que está detectando es un identificador o una palabra reservada.
- **Patrones:** Cada patrón es reconocido por un Autómata Finito Determinista, el problema es reconocer más de un patrón a la vez, pues en ese caso se utiliza la anticipación para saber si se trata de un patrón u otro.

Gramática abstracta y tabla de tokens

- **Gramática abstracta:** Es la gramática resultante de considerar los tokens como símbolos terminales y eliminar aquellas producciones en la que derivan los tokens.
- **Tabla de símbolos:** Es el lugar de almacenamiento temporal (durante la fase de compilación y, en lenguajes orientados a objetos, durante ejecución) debido a la necesidad de identificar los lexemas en la ocurrencia de cada token durante las fases posteriores de traducción.
- **Tabla de tokens:** Tabla formada por tantas filas como tokens se haya identificado, las columnas tienen esta información: token, código, atributos y patrón (regex).

2.4 Tratamiento de errores:

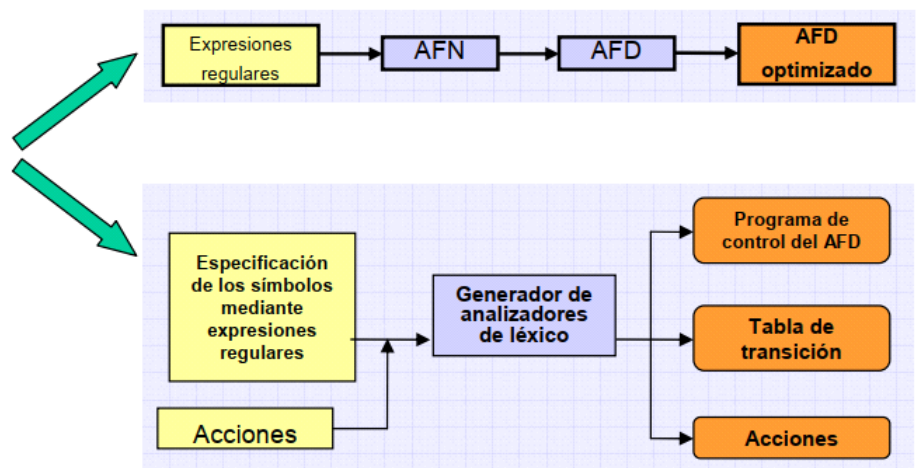
- Un traductor debe adoptar alguna estrategia para detectar, informar y recuperarse para seguir analizando hasta el final.
- Las respuestas del traductor pueden ser:
 - **Inaceptables:** Provocadas por fallos del traductor, entrada en lazos infinitos, producir resultados erróneos y detectar solo el primer error y detenerse.
 - **Aceptables:** Evitar la avalancha de errores (mala recuperación) y, aunque más complejo, informar y reparar el error automáticamente.
- El comportamiento de un analizador léxico es el de un escáner o Autómata Finito.

- Un error léxico es cualquier carácter no reconocido, es decir, cuando el analizador detecta que no existe transición desde el estado que se encuentra para el símbolo de entrada: el símbolo de entrada no era el esperado.
- Estrategias que se pueden adoptar:
 - Pasar al estado inicial ignorando los símbolos previos del lexema, iniciando de nuevo el proceso de identificación de tokens.
 - Proceso de sincronización, también conocido como modo pánico (panic). Consiste en ignorar los símbolos de la entrada no esperados hasta que aparezca un símbolo esperado, esta es una buena estrategia cuando aparecen símbolos extraños de forma extra.
 - Reparar los errores mediante la transformación de lexemas (borrar caracteres extraños, insertar caracteres que falten, intercambio o sustitución de caracteres).

2.5 LEX:

- Es un generador de analizadores de léxico
 - Permite construir analizadores de léxico como programas independientes.
 - Permiten construir la rutina léxica que utiliza el YACC para generar analizadores sintácticos.
 - Permiten hacer programas que realicen análisis, estadísticas o cualquier tipo de transformación a nivel léxico en un texto fuente.

Implementar el
analizador de léxico



2.5.2 Especificación LEX

```
{Definiciones}
```

```
%%
```

```
{Reglas}
```

```
%%
```

```
{Procedimientos del usuario}
```


2.5.2 Especificación LEX: Definiciones

Pueden contener:

- De forma opcional, **código C** con el siguiente formato:

```
%{  
                                /* código C */  
%}
```

- **Definición** de un nombre mediante una **expresión regular** con la forma:

Nombre	expresión_regular
--------	-------------------

Ejemplo:

letra	[a-zA-Z]
digito	[0-9]

NOTA: Su posterior uso requerirá que vaya el nombre entre llaves, es decir, se tendrían que usar del siguiente modo: {letra} ó {digito}

2.5.2 Especificación LEX: Reglas

Esta parte de la especificación **LEX** contiene los patrones que representan todos los símbolos a reconocer por el analizador de léxico, así como las acciones a realizar cuando son identificados.

Patrón (expresión regular)	{ Acción }
----------------------------	------------

Ejemplo:

letra	[a-zA-Z_]
digito	[0-9]

%%

...

{letra}({digito} {letra})*	{ return ID; }
----------------------------	----------------

...

%%

2.5.3 Consideraciones finales: Acciones

Las acciones representan la consecuencia de que el analizador de léxico haya reconocido un determinado lexema asociado a un patrón. El formato de las acciones es el siguiente:

<Acción>	::=	<Acción_Simple> "{" <Acción_Compuesta> "}"
<Acción_Simple>	::=	";" <Acción_Especial> ";" <sentencia_C> ";" " "
<Acción_Compuesta>	::=	<Acción_Simple> { <Acción_Simple> }
<Acción_Especial>	::=	ECHO REJECT BEGIN

Acciones por defecto

- Para los caracteres **reconocidos** → No realiza ninguna acción.
- Para los caracteres **no reconocidos** → Copiarlos en el fichero de salida (stdout).

Acciones ; y |

- ; Representa la **acción por defecto** para los caracteres reconocidos.
- | Indica que la acción asociada al patrón es la **misma** que la del **siguiente patrón**.

Acciones especiales

- **ECHO**: Copia la secuencia de caracteres reconocidos en el fichero de salida.
- **REJECT**: Se usa para las reglas ambiguas.
- **BEGIN**: Útil cuando se desea tener construcciones sensibles al contexto por la izquierda.